Counting the Cold: Team Build Plan

Event: Reboot Earth hackathon, UC Davis, Oct 23–24 2026, 24 hours · **Team:** 3 people · **Challenge:** "Counting the Cold" (UC Davis FNRIC) **Repo:** https://github.com/Chuckisawsome12345/counting-the-cold

> **The live, editable plan is the team doc: https://claude.ai/artifact/RircwznHknoxj1voHwgYpK.** This file (and `docs/BUILD_PLAN.pdf`) is the kickoff snapshot from Oct 23. If they disagree, the live doc wins.

> **How to use this doc**
>
> - **Humans:** read §0 (onboarding) and §8 (your role), then pick up issues labeled with your role.
> - **Claude / AI assistants:** read §12, "Rules for AI assistants", before doing anything. Tell your assistant which role you are (A, B, or C).

Context

One-liner: "*Your orchard's chill and heat, not a station's.*"

California fruit and nut growers track two temperature metrics:

- **Chill:** winter cold, accumulated from hourly temperatures. It predicts bud break.
- **Growing degree days (GDD):** heat, accumulated from daily max and min. It predicts pests, disease, and harvest.

Growers rely on CIMIS weather stations, but those are often far away, sit in a different microclimate, or have closed. For example, Durham #12 in Butte County closed in July 2025.

The product: a free web app. A grower drops a pin on their orchard and sees:

1. Chill and GDD for the season so far
2. A comparison to the last 10 seasons
3. A 7–14 day projection

It works anywhere, with no hardware.

Differentiator: we validate gridded weather data against CIMIS stations, show the error, and apply a bias correction to orchard estimates. The demo shows a Butte County orchard near the closed Durham station still getting numbers.

Facts verified Oct 23 2026 that shape this plan:

- **The CIMIS API is flaky.** `/api/station` returned `E90002: Unhandled server exception` for part of the evening, then recovered. So we hardcode station metadata (`backend/data/stations.json`, pulled from the API) and pull CIMIS data early into committed files. The running app never calls CIMIS.
- **Durham #12 disconnected on 2025-07-30** (records from 1982). The nearest *active* stations are **Biggs #244** (25 km, since 2015), **Williams #250** (53 km, since 2016), and **Gerber South #222** (56 km, since 2014). Orland, Colusa, and Nicolaus are all closed.
- **Open-Meteo archive works.** Hourly °F data is returned for any lat/lon with `timezone=GMT`.
- **FNRIC counting windows:** Dynamic Model chill portions count Sep 1 through Aug 31. Chill hours count Nov 1 through the end of February.

- **The FNRIC almond hull split model uses GDD90**: degree days from full bloom to 90 days after full bloom, single-sine method (Tombesi et al. 2010). It does not use degree hours.

Team decisions

Topic	Decision
Repo	Public GitHub repo. Main is protected, so every change goes through a PR.
Team	3 roles (A, B, C), each owning separate folders (§8). People self-assign issues by role label.
Seasons	A season runs Sep 1–Aug 31 (label "2025-26"). Portions start Sep 1, chill hours and Utah start Nov 1, GDD starts Jan 1 or at biofix. All start dates are configurable. The dashboard opens on **2025-26**, with a toggle to 2026-27 (portions already accumulating plus the forecast).
Units	°F everywhere a user sees it. The Dynamic Model converts to Kelvin internally.
Time	The internal hourly index uses **fixed PST (UTC–8, no daylight saving)** to match CIMIS. Open-Meteo is fetched in GMT and shifted. CIMIS hour `2400` becomes 00:00 the next day.

---

0. Onboarding (each teammate, about 10 minutes)

1. Accept the GitHub collaborator invite and clone the repo.
2. Pick a role: **A (Data & API)**, **B (Science & Validation)**, or **C (Frontend & Demo)** (§8).
3. Create your local env file: `cp .env.example .env`.
   - Only people who run the CIMIS fetcher (B) or deploy the backend (A) need `CIMIS_KEY`.
   - Get a free key from CIMIS (et.water.ca.gov) and put it in `.env`.
   - **Never commit it, paste it in chat, or put it in an issue.**
4. Start the backend:

   ```
   cd backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt &&
   uvicorn app.main:app --reload
   ```

5. Start the frontend: `cd frontend && npm install && npm run dev`. It proxies `/api` to `localhost:8000`.
6. Claim your work: `gh issue list --label role:<data-api|science|frontend>`, then `gh issue edit <N> --add-assignee @me`.

1. Tech stack

FastAPI (Python) + React/Vite (TypeScript) + Leaflet + Recharts. Streamlit would get to working faster, but its pin-dropping and chart interactions look clunky in a demo. The split stack also lets 3 people work in parallel without touching each other's files.

- **Backend:**
  - Python 3.12+, FastAPI, uvicorn, httpx, pandas, numpy, pydantic v2, python-dotenv, pytest.
  - The cache is a plain SQLite file through the `sqlite3` module, with no ORM.
- **Frontend:**

  - Vite with the react-ts template, react-leaflet with OpenStreetMap tiles, Recharts, and plain CSS (or Tailwind if C prefers).
  - TypeScript runs with `strict: false`. TS is only there so `types.ts` acts as the contract with the backend.
- **Infra:** GitHub Actions CI (pytest + `npm run build`), Render for the backend, Vercel for the frontend.

2. Folder structure

The owner letter (A, B, C) for each path is in parentheses.

```
counting-the-cold/
├── README.md                    # setup + run (A)
├── .gitignore  .env.example     # CIMIS_KEY=, DEMO_MODE=0, ALLOWED_ORIGINS=
├── .github/
│   ├── workflows/ci.yml         # backend-tests + frontend-build, always run (no path filters)
│   └── pull_request_template.md
├── docs/
│   ├── BUILD_PLAN.md            # this plan
│   ├── CONTRIBUTING.md          # git workflow + folder ownership table
│   ├── api-contract.md          # source of truth for JSON shapes (shared, see §8)
│   └── demo-script.md           # (C)
├── backend/
│   ├── requirements.txt         # all deps added at scaffold time (A)
│   ├── app/
│   │   ├── main.py  config.py  schemas.py          # (A)
│   │   ├── routes/   point.py stations.py validation.py meta.py   # (A)
│   │   ├── sources/  openmeteo.py cache.py snapshots.py  # (A); cimis.py (B, offline scripts only)
│   │   ├── pipeline/ build_point.py timeseries.py                # (A) orchestration
│   │   ├── calc/     chill.py gdd.py seasons.py gaps.py projection.py hullsplit.py crops.py  # (B)
│   │   └── validation/ bias.py metrics.py                         # (B)
│   ├── scripts/
│   │   ├── fetch_cimis.py       # (B) resumable download → data/cimis/
│   │   ├── build_validation.py  # (B) → data/validation.json
│   │   └── prewarm.py           # (A) demo pins → cache/snapshots
│   ├── data/
│   │   ├── stations.json        # hardcoded metadata incl. Durham active=false (A)
│   │   ├── cimis/               # station-season JSON.gz (B)
│   │   ├── snapshots/           # demo-pin responses for DEMO_MODE (A)
│   │   └── validation.json      # (B)
│   └── tests/                   # calc + validation tests (B); test_api, test_sources (A)
│       └── fixtures/fnric_reference.json  cimis_sample.json
└── frontend/                    # everything here is (C)
    ├── vite.config.ts           # proxy /api → localhost:8000
    └── src/
        ├── api/   client.ts types.ts mock/point.json
        ├── hooks/ usePoint.ts
        └── components/ MapPicker SettingsBar SummaryCards SeasonChart
                        ForecastChart ValidationPanel DataQualityBadge
                        CorrectionInfo StationCompare HullSplitCard
```

3. Data pipeline and caching

Request flow for `/api/point`. A orchestrates and calls B's pure functions.

1. **Snap** lat/lon to 0.1°, the ERA5-Land grid size, so nearby pins share cache entries. The UI still shows the exact pin.
2. **Fetch** hourly `temperature_2m` in °F, GMT, from two places:

    - The archive API (`archive-api.open-meteo.com/v1/archive`), from Sep 1 ten seasons back through today − 6 days. The archive lags about 5 days.
    - The forecast API (`api.open-meteo.com/v1/forecast`), with `past_days=10&forecast_days=16`.
3. **Merge:** archive data wins where both overlap. The forecast fills the recent days and the future. Each row gets a `source` tag.
4. **Normalize:**
    - Convert to PST and run `calc.gaps.fill_gaps`.
    - Apply the bias correction (see the Differentiator section).
    - Derive daily tmax and tmin from the hourly data, using PST days.
5. **Compute and return:** run the models for each season, plus history, forecast, and climatology, then return the response.

CIMIS is offline-only. Only `scripts/fetch_cimis.py` touches it, never the request path.

- **Stations:**
    - 6 Davis, 70 Manteca, 71 Modesto, 39 Parlier, 5 Shafter.
    - **12 Durham**, using its history through 2025-07-30.
    - The 3 nearest active stations to Durham, so the Butte County correction has nearby neighbors: **244 Biggs**, **250 Williams**, **222 Gerber South**.
- **Requests:**
    - Hourly `hly-air-tmp` and daily max/min, one station-season per request. If a request is refused, fall back to monthly chunks.
    - Auth header: `Ocp-Apim-Subscription-Key`.
    - JSON path to the records: `Data > Providers > Records`.
- **Resilience:** retry with backoff, and write each station-season to its own file so a crash or outage resumes where it left off.
- **QC flags:** blank means good. `Y` is kept but flagged. `M`, `R`, `Q`, `S`, and `N` are treated as missing.
- **Storage:** downloaded files are **committed** to `data/cimis/`.

Caching layers:

Layer	What	TTL
L1 SQLite (`backend/.cache/http.sqlite`, gitignored)	Raw upstream responses (gzip), keyed by source + normalized params	Archive and past CIMIS seasons forever. Forecast 1 h.
L2 in-memory LRU	Computed `/api/point` result per (snapped cell, settings)	15 min
L3 committed snapshots (`data/snapshots/*.json.gz`)	Full responses for demo pins and stations	Used when `DEMO_MODE=1`, or when a live fetch fails and there is no cache entry

Fallback chain:

1. L2 in memory
2. L1 fresh entry
3. Live fetch
4. L1 stale entry. The response carries `stale: true` and the UI shows a banner.
5. L3 snapshot

6. 503 with a clear message

Rate budget: Open-Meteo's free tier allows about 600 calls/min, 5k/hour, and 10k/day. It **counts long, multi-week requests as several calls** (confirm on their terms page). Ten years of hourly data for a new pin is expensive, which is why we snap, cache forever, and run `prewarm.py` at startup.

4. Calculation module (B): pure functions, no I/O

All inputs are pandas Series indexed by PST timestamps, in °F. NaN means missing.

```
# calc/chill.py
chill_hours(t: Series, lo=32.0, hi=45.0) -> Series        # 1 if lo <= t <= hi, else 0; NaN stays NaN
utah_units(t: Series) -> Series                            # Richardson bands, °F:
    # <=34:0 | 35–36:0.5 | 37–48:1 | 49–54:0.5 | 55–60:0 | 61–65:-0.5 | >65:-1
    # season cumulative is floored at 0 (document the choice, toggleable)
dynamic_portions(t: Series) -> Series                      # cumulative portions (Fishman/Erez)
    # constants: e0=4153.5 e1=12888.8 a0=139500 a1=2.567e18 slp=1.6 tetmlt=277
    # chillR algorithm: TK=t_C+273; xi=sr/(1+sr) with sr=exp(slp*tetmlt*(TK-tetmlt)/TK);
    # xs=(a0/a1)*exp((e1-e0)/TK); ak1=a1*exp(-e1/TK); intermediate E loops over hours;
    # when E>=1 a portion of E*xi is banked. Missing hours: carry state, skip increment.
# calc/gdd.py
daily_gdd(tmax, tmin, base, upper=None, method="average"|"single_sine") -> Series
    # average = max(0, (tmax+tmin)/2 - base), with horizontal cutoff at upper
    # single_sine (Zalom 1983) is needed for hull split, and UC IPM uses it
# calc/crops.py   presets: almond-NOW 55/94°F, walnut-codling-moth 50/88°F, generic 50/none, custom
# calc/seasons.py season_window(season: int, start_mmdd) -> (start, end); handles Feb 29
# calc/gaps.py    fill_gaps(t, max_interp_h=3) -> (filled, GapReport{missing, interpolated, long_gaps[]})
# calc/projection.py  forecast continuation + analog-season p10/p50/p90 end-of-season
# calc/hullsplit.py   (stretch) GDD90 from bloom date, coefficients from Tombesi 2010
```

Tests run with `pytest`, all offline, in CI.

- **Hand cases:**
  - Chill hours: inputs [31, 32, 40, 45, 46] give [0, 1, 1, 1, 0].
  - Utah: every band boundary.
  - GDD: tmax 80, tmin 50, base 50 gives 15. Temperatures below base give 0. The upper cutoff works.
  - Seasons: windows in leap years.
- **Properties:**
  - Dynamic Model portions never decrease.
  - A constant 80°F series gives about 0 portions.
  - Gaps of 3 h or less get interpolated. A 5 h gap is reported in `long_gaps`.
- **Bias:** a synthetic series where gridded = observed − 2°F gives a +2 table, and applying the table returns the observed series.
- **Golden values against FNRIC:**
  - In H0–H2, B records 4–6 published values in `tests/fixtures/fnric_reference.json`. Each entry has station, season, model, end date, value, and URL.
  - Source: FNRIC's "Cumulative Chilling Portions" pages for Davis #6, Parlier #39, and Shafter #5, recent seasons.
  - The test recomputes each value from the committed CIMIS files and asserts a match within ±1 portion or ±2%.
  - FNRIC substitutes a backup station when data is missing, which explains small differences.
  - If anyone has R, also cross-check `chillR::Dynamic_Model` on `cimis_sample.json`.

5. API endpoints (A)

FastAPI serves auto-generated docs at `/docs`.

Method	Path	Returns
GET	`/api/health`	`{ok, cache_entries, demo_mode}`. Used for warm-up pings.
GET	`/api/point?lat&lon&season=2025&crop=almond&portions_start=09-01&hours_start=11-01&gdd_start=01-01&corrected=true`	Everything the dashboard needs (shape below)
GET	`/api/stations`	`stations.json` plus each station's validation summary
GET	`/api/validation`	Raw vs corrected error per station, season, and metric, plus overall MAE
GET	`/api/crops`	Crop presets
GET	`/api/compare?lat&lon&season`	*Stretch:* pin vs nearest station, side by side
GET	`/api/hullsplit?lat&lon&bloom=YYYY-MM-DD&cultivar=`	*Stretch*
GET	`/api/v1/models?lat&lon`	*Stretch:* public open API with totals for every model

The `/api/point` response shape below is frozen in `docs/api-contract.md` at H0 and mirrored in `schemas.py` and `types.ts`.

```
{
  "location":   {"lat":39.61,"lon":-121.82,"grid_lat":39.6,"grid_lon":-121.8},
  "season":     {"label":"2025-26","portions_start":"2025-09-01","hours_start":"2025-11-
01","gdd_start":"2026-01-01","as_of":"2026-10-23"},
  "totals":     {"chill_portions":62.1,"chill_hours":812,"utah_units":1104,"gdd":3950},
  "daily":      [{"date":"2025-09-
01","tmin":55.2,"tmax":91.0,"chill_portions":0,"chill_hours":0,"utah_units":0,"gdd":0,"source":"archive","f
illed":false}],
  "forecast":   {"daily":[/* same shape, source:"forecast" */],"plus7":{/*totals*/},"plus14":{/*totals*/}},
  "history":    [{"season":"2024-25","totals":{},"cumulative":[{"day":0,"chill_portions":0,"gdd":0}]}],
  "climatology":{"chill_portions":{"p10":55,"p50":63,"p90":70}},
  "quality":    {"missing_hours":12,"interpolated_hours":10,"long_gaps":
[{"start":"...","end":"...","hours":6}],"stale":false},
  "correction": {"applied":true,"stations":
[{"id":12,"name":"Durham","active":false,"distance_km":3.1,"weight":0.7}],
                 "expected_error":{"chill_portions":1.8,"chill_hours":40,"gdd":35}},
  "nearest_station": {"id":12,"name":"Durham","active":false,"distance_km":3.1}
}
```

The numbers above are placeholders that show the shape, not real results.

Differentiator: validation and bias correction (B builds the math, A wires it in)

Bias model: for each station, a 12 × 24 table of the mean difference (CIMIS − gridded) in °F, by month and hour of day, fit on hourly data. It's simple and explainable in one sentence.

Correcting an orchard pin:

- Blend the station tables by inverse-distance weighting (power 2, stations within 150 km).
- If a **closed station within 10 km has at least 3 seasons of history**, weight its own table heavily. For the Durham pin, the demo line is: "*the station closed, but its decade of records still calibrates our estimate.*"
- The response lists which stations were used and their weights.

Honest validation (`build_validation.py` writes `validation.json`):

- For each station and season, compute every metric three ways: from CIMIS (the truth), from raw gridded data, and from corrected gridded data.
- The correction is **leave-one-station-out**: station X is corrected using only the other stations' tables. That's the same situation an orchard pin is in.
- Report MAE per metric, raw vs corrected.
- Durham's history through 2024-25 is the headline validation case.

Model check (cheap): the script also scores Open-Meteo's `best_match`, `era5_land`, and `ecmwf_ifs` models. We switch to whichever has the lowest error.

6. UI (C): one-page dashboard

- **Header:** name and one-liner, plus two tabs: **Orchard** (default) and **How accurate is this?**
- **Map (left):**
  - Click to drop a pin.
  - Station markers: active stations filled; closed stations as a hollow grey marker with a "Durham, closed Jul 2025" tooltip.
  - Demo shortcut buttons: *Butte orchard near Durham*, *Davis*, *Parlier*.
  - *Stretch:* address search via Nominatim, kept to low volume.
- **Settings bar:** season dropdown, crop preset, the three start dates, and a *bias-corrected* toggle.
- **Summary cards:** Chill Portions, Chill Hours, Utah, and GDD. Each card shows the value, ▲/▼ vs the 10-season median, the +14-day projection, and the expected ± error from validation.
- **Season chart** (Recharts ComposedChart):
  - Cumulative value vs day of season.
  - Past seasons as faint grey lines, the median dashed, the current season bold.
  - The forecast as a dotted line for 14 days, then a p10–p90 band to the end of the season.
  - Tabs to switch between the four metrics.
- **Season totals bar chart** for the last 10 seasons.
- **DataQualityBadge:** counts of missing, interpolated, and long-gap hours, with tooltips. Also handles the stale-data banner.
- **CorrectionInfo:** a line like "Corrected using Durham (historical) 70%, Orland 20%, Davis 10%."
- **Validation tab:** a per-station table and bars comparing raw vs corrected error, with a Durham callout.
- **Stretch:** StationCompare panel (pin vs nearest station) and HullSplitCard.
- **States:** loading skeletons, plus error and stale banners. C builds against `mock/point.json` until the live API lands, around H4.

7. Milestones (H = hours from kickoff; working end to end by H4)

When	Milestone	Done means
H0–H1	**M0 Setup**	Repo, scaffold, contract, mock JSON, CI, and issues exist (§13). Everyone has cloned and runs both apps locally (§0).
H1–H4	**M1 Walking skeleton**	A dropped pin returns real Open-Meteo temps, and chill hours and GDD show in cards. **Deployed** to Render + Vercel.
H4–H8	**M2 Core science**	Utah and Dynamic Model with golden tests, gap fill, L1/L2 cache, 10-season history, forecast merge. Season chart with history and forecast, plus the settings bar.
H8–H13	**M3 Differentiator**	CIMIS pulled for all stations, `validation.json` built, correction wired in, validation tab, station markers, Durham preset.
H13–H17	**M4 Projection and polish**	p10/p50/p90 projection, prewarm and snapshots with `DEMO_MODE`, error and stale handling, responsive layout.
H17–H20	**M5 Stretch**	Pin vs station compare, then hull split, then public `/api/v1` docs. Start only after M4 is green.
H20–H24	**M6 Freeze and demo**	**Feature freeze at H20.** Final deploy and snapshot commit at H21. Rehearse the demo three times, record a backup video, write the Devpost entry and README.

Suggested sleep shifts (about 3 hours each, so someone is always awake to merge): C sleeps H11–H14, B H14–H17, A H17–H20.

8. Roles and conflict prevention

Role	Owns (only this person edits)	Core issues
A: Data & API (strongest backend person)	`backend/app/{main,config,schemas}.py`, `routes/`, `sources/` (except `cimis.py`), `pipeline/`, `scripts/prewarm.py`, `data/stations.json`, `data/snapshots/`, `requirements.txt`, root files, Render deploy	Open-Meteo client and merge, PST normalization, SQLite cache, `/api/point` skeleton, history and forecast assembly, correction wiring, stations and validation routes, snapshots and DEMO_MODE, deploy, README
B: Science & Validation (strongest Python and math person)	`backend/app/calc/`, `backend/app/validation/`, `backend/tests/` (except test_api and test_sources), `sources/cimis.py`, `scripts/fetch_cimis.py`, `scripts/build_validation.py`, `data/cimis/`, `data/validation.json`, test fixtures	FNRIC reference values; chill hours, Utah, GDD, and seasons **by H2** (this unblocks A); Dynamic Model and golden tests; gaps; CIMIS fetcher (start early, the API is flaky); bias model and IDW; leave-one-out validation; projection; hull split
C: Frontend & Demo	`frontend/**`, `docs/demo-script.md`, Devpost text, slides, Vercel deploy	Map and pins, cards, season chart, settings, quality badge, validation tab, correction info, compare panel, demo script, backup video

Rules that prevent overwriting each other's work:

1. **The scaffold commit creates every shared file before anyone starts.** That covers root config, CI, folder skeletons, stub modules with final function signatures, `api-contract.md`, and `mock/point.json`. No two people ever create the same file.
2. **Only edit files in your own column.** If you need something from another area, comment on that owner's issue or ping them.
3. **Contract changes:** open a PR labeled `contract` that edits `docs/api-contract.md`, and @mention both other teammates. A then mirrors the change in `schemas.py`, and C mirrors it in `types.ts`.
4. **Lockfiles:** only C touches `package-lock.json`, and only A touches `requirements.txt`. Every dependency B needs is added at scaffold time.
5. **Branches and PRs:**
   - One branch per issue, prefixed by role: `data/12-openmeteo-merge`, `sci/7-dynamic-model`, `ui/20-season-chart`.
   - Each PR says `Closes #N`.
   - Run `git pull --rebase origin main` before opening a PR.
   - Keep PRs small and merge every 1–2 hours.
6. **GitHub enforces the workflow:**
   - A ruleset on `main` requires a PR, requires the `backend-tests` and `frontend-build` checks to pass, and blocks force pushes and branch deletion.
   - Required approvals are 0, for speed.
   - Only squash merges are allowed, and branches are deleted automatically after merge.
7. **Secrets:**
   - `.env` is gitignored.
   - `CIMIS_KEY` lives only in local `.env` files and in Render's environment variables.
   - The key is used server-side only and never reaches the frontend.

9. Risks and fallbacks

Risk	Fallback
CIMIS API down (it was on Oct 23) or refusing large requests	Station metadata is hardcoded. The resumable fetcher retries and falls back to monthly chunks. Data is committed once fetched. The running app doesn't need CIMIS.
Open-Meteo down or rate-limited	Snapping and caching, prewarmed demo pins, stale cache served with a banner, L3 snapshots, `DEMO_MODE=1` during judging
Archive lags about 5 days	Merge in the forecast API's `past_days`
Gridded data (about 9–11 km) misses orchard microclimates	Bias correction plus a visible expected error. PRISM 800 m data can be added later as a validation layer.
Time zone or hour misalignment (CIMIS uses PST and hour `2400`)	A single normalization function with a unit test
Missing or flagged data	Interpolate gaps of 3 h or less. Flag longer gaps in the UI with counts. Gridded data can fill CIMIS gaps during validation.
Dynamic Model bug	Golden tests against FNRIC and chillR
Render free-tier cold start (about 50 s)	Ping `/api/health` 2 minutes before judging. Railway as backup.

Risk	Fallback
Venue Wi-Fi fails	Run locally with `DEMO_MODE=1` (no network needed), plus the backup video
Merge conflicts	Folder ownership, scaffold-first, small PRs
Scope creep	Stretch goals only after M4. Hard freeze at H20.

10. Deployment

- **Backend on Render** (free web service):
  - Root directory: `backend`. Build command: `pip install -r requirements.txt`.
  - Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`.
  - Environment variables: `PYTHON_VERSION=3.12.x`, `CIMIS_KEY`, `ALLOWED_ORIGINS=<vercel url>`, `DEMO_MODE`.
  - The disk is ephemeral, so committed snapshots plus a prewarm at startup cover the demo.
- **Frontend on Vercel:** root `frontend`, Vite preset, `VITE_API_URL=<render url>`. Every PR also gets a preview URL.
- **Deploy schedule:** deploy the skeleton at H4, not at the end. Pushes to main auto-deploy both apps. The final tagged deploy is at H21.
- **Backups:** Railway for the backend, a local laptop running `uvicorn` and `npm run preview`, and the recorded video.

11. Demo script (about 2:30)

1. **0:00 Hook.** "In July 2025, CIMIS station 12 in Durham, Butte County, shut down. Every almond grower who used it lost their chill count for last winter."
2. **0:20 Drop a pin** on a Butte orchard near Durham. The 2025-26 cards appear: portions, hours, Utah, GDD. "No station, no hardware. Your orchard's chill and heat, not a station's."
3. **0:50 History chart.** This season vs the last 10 and the median. "Chill finished X% above normal."
4. **1:10 Toggle to 2026-27.** Portions have been accumulating since Sep 1. The 14-day forecast runs into November, with an end-of-season range.
5. **1:30 Trust: the validation tab.** "We checked ourselves against 9 CIMIS stations. Raw gridded data was off by X portions; corrected, it's off by Y. And Durham's decade of records still calibrates the estimate after the station closed."
6. **2:00 Microclimate.** Show the pin and the nearest station side by side (stretch), or move the pin and watch the numbers change.
7. **2:15 Open and free.** Show `/docs`, the open API. Close on the one-liner.

12. Rules for AI assistants (Claude Code etc.) working in this repo

Your human tells you their role: A, B, or C. Then:

1. **Read first:** `docs/BUILD_PLAN.md` (this file), `docs/CONTRIBUTING.md`, and `docs/api-contract.md`.
2. **Only create or edit files your role owns** (§8 table). If a task needs a change elsewhere, stop and tell your human who owns it. Don't make the change yourself.

3. **Never change the shape of `docs/api-contract.md`, `backend/app/schemas.py`, or `frontend/src/api/types.ts` unilaterally.** Contract changes go through a `contract` PR (§8 rule 3).
4. **Don't add dependencies** unless your role owns that lockfile or requirements file. Ask the owner instead.
5. **Git workflow:**
   - Work on a branch named `<data|sci|ui>/<issue#>-<slug>`. Rebase on `origin/main` before pushing.
   - Open a small PR with `Closes #N`.
   - Never push to `main`, force-push a shared branch, or rewrite another person's commits.
6. **Never read, print, commit, or log the CIMIS key** or the contents of `.env`. Use `.env.example` for placeholders.
7. **Keep the app working offline-safe:**
   - No CIMIS calls on the request path.
   - Every upstream call goes through `sources/cache.py`.
   - Respect `DEMO_MODE`.
8. **B's calc functions stay pure** (no I/O) and are covered by offline tests. Run `pytest` (backend) or `npm run build` (frontend) before opening a PR.
9. **Respect the milestone gate:** don't start `stretch` issues until M4 is done, or after the H20 freeze.

13. One-time repo setup (done once, by the repo owner)

1. **Scaffold** the folder (`git init`):
   - Root files, CI, PR template, and `docs/`: this plan, CONTRIBUTING with the ownership table, api-contract, and a demo-script stub.
   - A FastAPI skeleton whose `/api/point` returns the mock.
   - The frontend: `npm create vite@latest frontend -- --template react-ts`, plus react-leaflet and Recharts.
   - Stub modules with the §4 signatures, raising `NotImplementedError`.
   - `stations.json` with the 9 stations above, coordinates taken from the CIMIS station API.
   - `.env.example`.
2. **Check, then commit:** `pytest` collects tests, `uvicorn` serves `/api/health`, and `npm run build` passes. Then make the initial commit on `main`.
3. **Create the repo:** `gh repo create counting-the-cold --public --source . --push`.
4. **Settings:**
   - `gh repo edit` for squash-only merges and auto-deleted branches.
   - The `main` ruleset via `gh api repos/{owner}/counting-the-cold/rulesets`. It requires a PR and the checks `backend-tests` and `frontend-build`, blocks force pushes and deletion, and requires 0 approvals.
5. **Labels, milestones, and issues:**
   - Labels: `role:data-api`, `role:science`, `role:frontend`, `stretch`, `contract`, `bug`.
   - Milestones M0–M6.
   - About 30 issues from §7 and §8, each with a role label and milestone, left unassigned.
6. **Invite teammates:** `gh api -X PUT repos/Chuckisawsome12345/counting-the-cold/collaborators/<username> -f permission=push`. Once usernames are known, add `.github/CODEOWNERS` mapping folders to people (informational review requests).
7. **Share** the repo URL and `docs/BUILD_PLAN.pdf` with the team.

Verification

- **Scaffold:**
  - `cd backend && pytest` runs.
  - `uvicorn app.main:app` returns 200 from `/api/health` and the mock from `/api/point`.
  - `cd frontend && npm run build` passes, and `npm run dev` shows the app against the proxied mock.
- **GitHub:**
  - A direct `git push origin main` is rejected.
  - A test PR shows both required checks, and its branch is deleted automatically after a squash merge.
  - `gh issue list` shows the issues with role labels and milestones.
- **During the build:**
  - The FNRIC golden tests pass.
  - `/api/point` for the Durham pin returns 2025-26 totals, with `correction.stations` including Durham (historical).
  - `validation.json` shows corrected MAE below raw MAE.
  - `DEMO_MODE=1` works with Wi-Fi off.